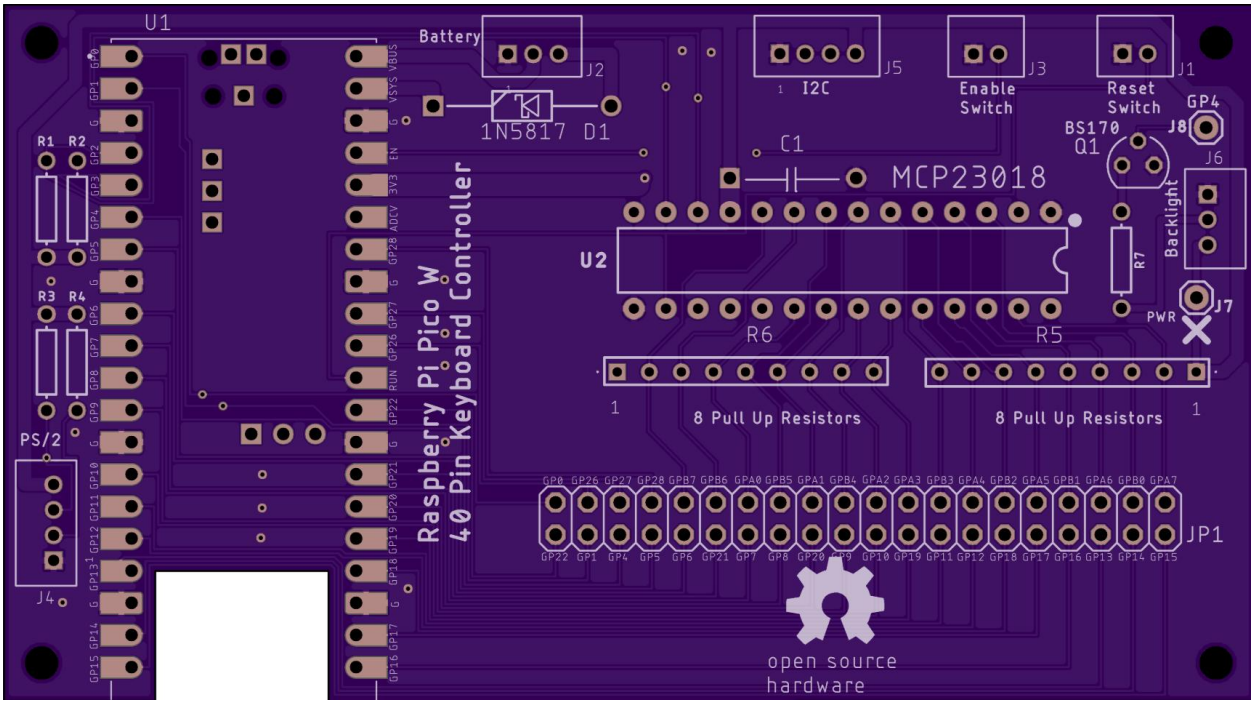
Raspberry Pi Pico W with Port Expander

I have designed a laptop keyboard controller board using a Raspberry Pi Pico (W) that includes an MCP23018 port expander. This increases the 26 pin GPIO limit of a stand-alone Pi Pico to 40 GPIO's. 16 GPIO's come from the port expander and 24 from the Pico with the I2C bus using the remaining 2 GPIO's. All components for this board including the FPC connector are through-hole for easy soldering. The Eagle board and schematic files along with a zipped Gerber layout file and software are at my repo.

The OSHPark rendition of the Raspberry Pi Pico W port expander board is shown below:

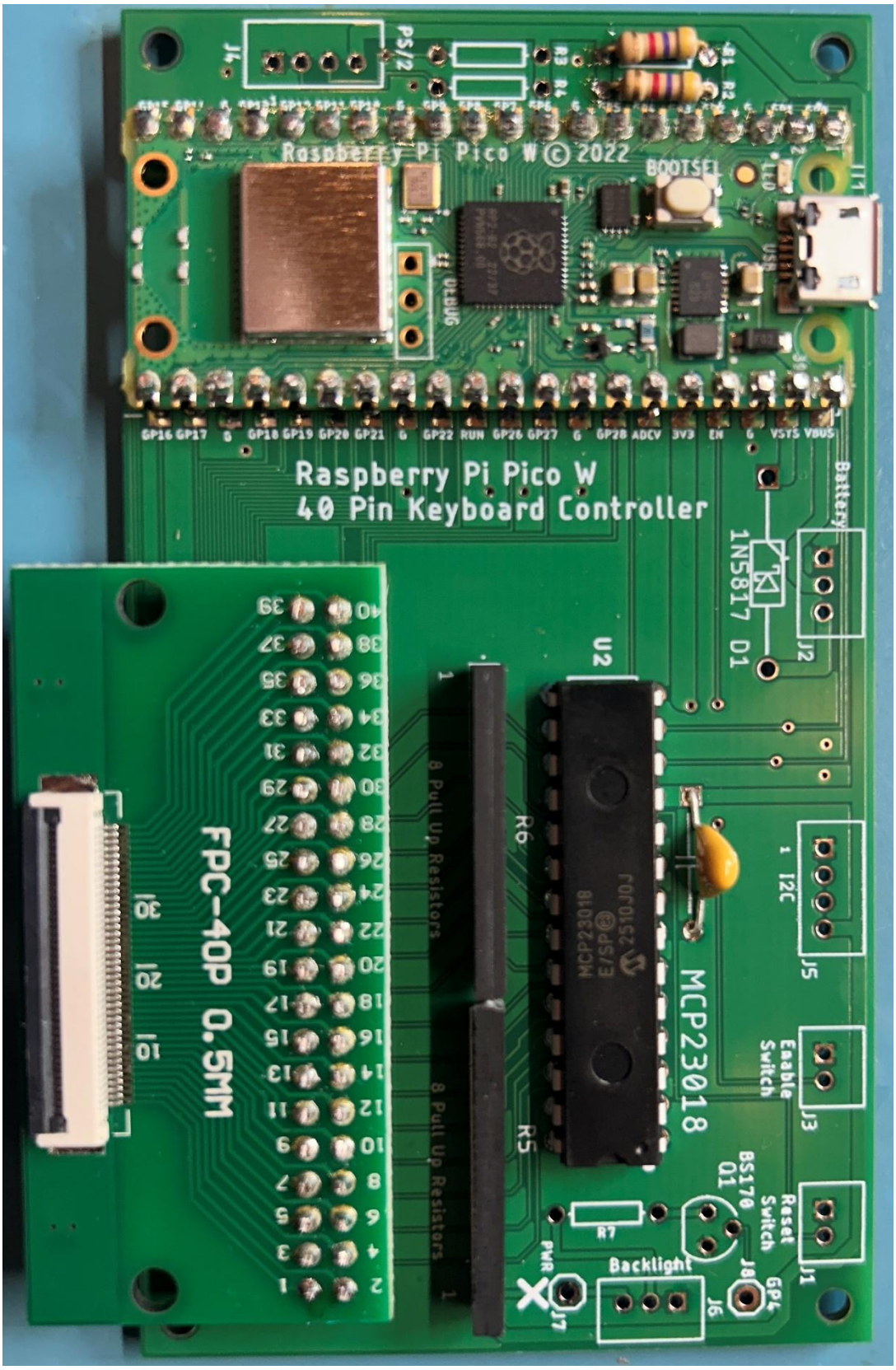
The size is 99mm x 55mm. JLCPCB.com cost is $6 for 5 boards (with slow shipping).



The cutout in the board gives better reception for the Bluetooth antenna on the Pico. The Pico can be mounted with header pins or soldered directly to the board for a lower profile.

The two rows of 20 pins at JP1 are for mounting an FPC adapter board. Choose an adapter board with the correct pitch and pin count that matches your keyboard's FPC cable. Don't get a board with pre-soldered header pins or you'll have to mount it underneath the Pico board. Solder header pins at JP1 with the adapter board sitting on top. Odds are your cable won't have 40 pins so install the FPC adapter board as far to the right as possible, leaving the pads on the left side empty. This will free up Pico GPIO pins for other uses such as touchpad and backlight control.

This picture shows the assembled board (without the extra items).



Bluetooth Low Energy (BLE) operation implies the controller will be running from a separate lithium battery with a charging circuit. USB 5 volts on the Pico VBUS pin is routed to pin 2 of a 3 pin JST connector at J2. This should be cabled (along with ground) to an off-board battery charger circuit. The nominal 3.7 volt battery output should be cabled to J2 pin 3. This feeds the anode of a 1N5817 Schottky diode at D1 that "or-ties" the battery voltage to the Pico's VSYS pin. When the USB cable is attached, a Schottky diode inside the Pico brings USB power to VSYS. If the keyboard will only be used for USB, it will always have power so Schottky diode D1 and the J2 connector are not needed.

The EN signal at Pico Pin 37 is routed to a 2 pin JST connector at J3. If you want to power down the Pico to save the battery, add a switch to connect the EN signal to ground.

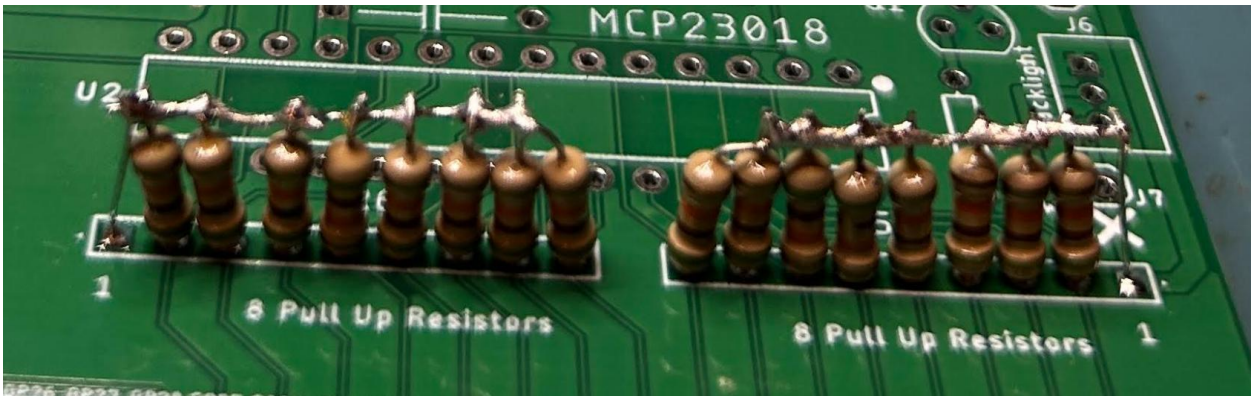
Resistors R1 and R2 are the I2C clock and data pullups and should be 4.7KΩ for running the bus at 400KHz. The I2C bus controls the MCP23018 port expander and it is also routed to the 4 pin JST connector at J5. This connector is only needed if interfacing to an I2C touchpad.

Clock and data pullup resistors R3 and R4 are 4.7KΩ and are driven by GPIO 0 and 1 of the Pico. These resistors and the 4 pin JST connector at J4 are only needed if interfacing to a PS/2 touchpad. Note that using these GPIO's for PS/2 reduces the available keyboard FPC pins to 38 instead of 40.

An additional keyboard pin can be repurposed for controlling a backlight LED by installing a 3 pin JST connector at J6. This connector provides 3.3 volts, ground, and an open drain N Channel FET with a current limit resistor for controlling an LED. Many laptop keyboards provide the backlight LED anode and cathode on multiple pins of a small FPC cable. Use a small FPC adapter board to wire the anode pins to the required voltage. There is 3.3 volts available at J6 pin 3 and 5 volts at Pico pin 40. The backlight cathode pins should go to J6 pin 2. Install a current limit resistor at R7 and a BS170 NFET at Q1. Size the resistor so the current at max brightness stays well under the Pico's 300ma limit. Pico GPIO 4 can be programmed to output a PWM signal to the gate of the NFET which turns on and off the high current to the backlight LED. It's best to test the backlight first with a bench power supply as shown in this PDF to see what voltage and current limit resistor works best.

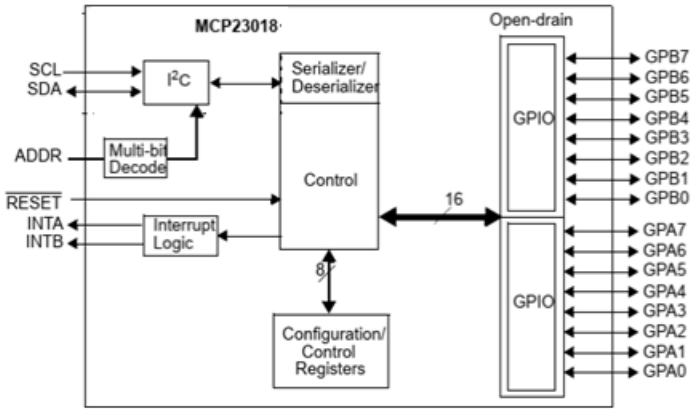
The MCP23018 Port Expander has 16 I/O pins. When programmed to be an output, it has open drain FETs instead of the typical totem pole configuration found in its sister part, the MCP23017. The MCP23018 can only drive low or float the output for a high, relying on a pull up resistor elsewhere in the circuit. Laptop keyboards do not have diodes on each switch so it's important to not create a sneak path to power when multiple keys are pressed. When programmed to be an input, there are internal pull up resistors that can be turned on in the code. Unfortunately, those resistors are about 100KΩ which is too high a value to quickly pull up the column pins of

the key matrix. I added pads on the circuit board for 16 pull up resistors. You can either use individual resistors as shown below or use two 9 pin pull up packages like the 4309R-101-473LF.



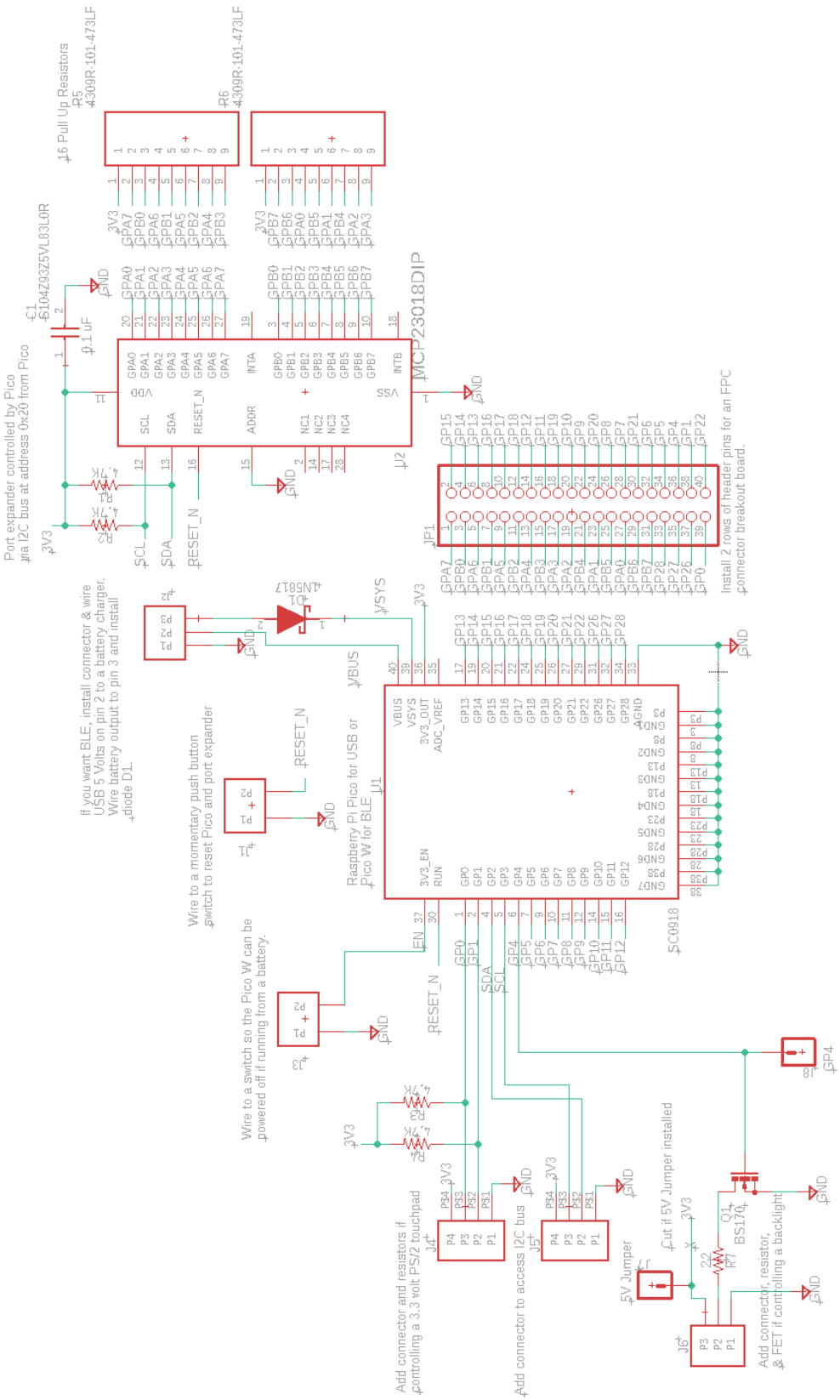
The resistor value determines how quickly the signal returns high when a key is released. You can add some delay in the code before reading the key matrix but too big a delay will be noticeable. The Pico has programmable pull up resistors that are about 50KΩ. I've done a lot of testing with Pico controlled keyboards and the 50KΩ pullup value has not caused any problems.

The MCP23018 port expander diagram is shown below.



My board powers this chip from the 3.3 volt regulator at Pico pin 36. The SDA and SCL signals are driven from the Pico I2C pins at GP2 and GP3. The 4.7KΩ pullup resistors (and short traces) allow the I2C bus to operate at 400KHz. The Address pin is grounded to set the I2C address for the port expander at 0x20. The MCP23018 RESET_n pin is tied to the Pico's RUN pin and to a 2 pin JST connector at J1 that can be wired to a push button reset switch. The INTA and INTB pins are not used. Each GP pin of the port expander can be programmed over I2C as an input or an output.

The Eagle Schematic is given below:



Programming – This board can be programmed for BLE operation using Arduino C++ code and the Earle Philhower board package. The Adafruit TinyUSB library is used to program Arduino C++ code for USB operation. QMK can also be used because it has modules for controlling the MCP23018 Port Expander and the keyboard backlight. To make KMK work with this board, the AI “Claude” wrote a new Python program that controls the port expander.

The MacBook Pro A1286 keyboard shown below was used for testing the port expander board and developing the BLE, USB, QMK, and KMK software. It has a 30 pin FPC cable with a 0.5mm pitch.



This keyboard has 30 FPC pins so I’ve slid the FPC cable to the far right of the 40 pin connector. All 16 GPIO’s from the port expander are used and the remaining 14 pins are connected to the Pico GPIO's.

Keyboard Decoding

Regardless of which code will be used to make your keyboard controller, you must first decode how the keyboard is wired. For this task, I developed Arduino C++ code for the Pico called “Pico_40Pin_Matrix_Decoder_USB.ino“ using the Adafruit TinyUSB library. It detects when a key is pressed and sends the two connected I/O pin numbers as text over USB. The PC that receives this text should have a list of keyboard keys open in an editor so each key’s information can be captured. A feature of this code is that it initially scans all 40 FPC pins and if any are shorted, they are removed from the scan list. It’s common to have ground pins or LEDs on the FPC cable and these pins need to be ignored. The result of running this code on the MacBook keyboard can be found at my repo. I’ve previously used this keyboard with a Teensy so I was able to check the results for accuracy. If you would rather use Circuit Python, the code called “matrix_decoder_mcp23018_40pin.py” runs on the Pico and acts just like the C++ code above.

Here are the steps using the TinyUSB decoder code:

- Load the Earle Philhower board package and the Adafruit TinyUSB Library into the Arduino IDE.
- Set the USB stack to Adafruit TinyUSB.
- Bring up an editor like Notepad++ on your PC and load a blank keyboard pin list with QMK key names or TinyUSB key names.
- Place the PC cursor to the far right of the first key in the list (after the tabs).
- Connect the keyboard's FPC cable to the connector on the board with the contacts facing down.
- Load the Pico_40Pin_Matrix_Decoder_USB.ino code into the Arduino IDE. Push and hold the button on the Pico while you plug in the USB cable. Compile and upload the code to the Pico.
- Wait for the LED on the Pico to turn on before pressing any keys. The program looks for shorted pins and removes them from the scan. You can turn on the serial monitor to see what it finds.
- Press and release each key. Use your PC's arrow keys to jump over keys you don't have.
- Save the completed pin list text file.

There are 3 ways to determine the row/column pins and build the matrix.

1. Follow the manual procedure described in steps 12 and 13 of my Instructable.
2. Load the Matrix_Builder code into Thonny on your PC so it can analyze the keyboard pin list and output the matrix and the row/column pins. This program is geared towards QMK but it still gives you all the information you'll need with some editing. TinyUSB needs all KC_NO replaced with 0 at the positions with no keys.
3. Ask AI to determine the Row & Column Pins and make a key matrix. Modify the AI prompt in step 12 of my Instructable for a Raspberry Pi Pico.

For the MacBook Pro A1286 keyboard, I manually converted the key connection list into the row/column matrix shown below. Out of the 24 Pico GPIO's, this keyboard used 12. They are denoted by GP followed by a number. The port expander pins are denoted with GPA or GPB corresponding to the A and B ports. Each port has 8 bits numbered 0 through 7. Even though this keyboard has a 30 pin FPC connector, only 25 pins are for the keyboard keys. The remainder are no-connects, grounds, or LED's.

MacBook Pro A1286 Key Matrix

GPIO #	GPA0	GP7	GPB5	GP8	GP17	GPB1	GP16	GPA6	GPB0
GPA1	q	1	a	z	F11	bckspc	F12	F1	
GP20	w	2	s	x	esc	tab	`	F2	
GPB4	e	3	d	c				F3	
GP9	y	6	h	n	/	space	=	F6	
GPA2	r	4	f	v			]	F4	
GP10					enter			Shift-R	
GPA3				GUI-L	0		GUI-R		
GP19	t	5	g	b	-	Fn	[	F5	
GPB3					F10	Alt-R			
GP11					p	Caps lk			
GPB2	u	7	j	m	;	quote		F7	
GP18	i	8	k	comma		left	right	F8	
GPA5	o	9	L	period	\	down	up	F9	
GP14									Shift-L
GPA7									Alt-L
GP15									Cntr-L

Now that you know the row/column pins and key matrix, pick one of the 4 keyboard controllers outlined below. The Arduino C++ code for USB and BLE as well as KMK and QMK all have the same basic methodology. The Pico will drive a single row pin low and read the pulled-up column pins. When a column pin reads low, the key in the matrix at the associated row/column location is sent out. This repeats with the next row until it has cycled through them all. After a few milliseconds, the process repeats.

All of the routines use the same I/O port numbering to accommodate the port expander. The Pico GPIO bits in the code are denoted as 0, 1, 4 – 22, 26 – 28. There are missing GPIO's because 2 and 3 are used for I2C and 23 – 25 are for internal Pico use only. In the port expander, the first 8 bits are GPA pins and denoted as 100 – 107 in the code. The second 8 bits are GPB pins and denoted as 108 - 115. The code can tell when it needs to control a port expander bit as a row or column by checking if it's greater than 30. This table maps the 100 – 115 bit numbering to the GPA and GPB registers in the port expander.

GPA0	GPA1	GPA2	GPA3	GPA4	GPA5	GPA6	GPA7	GPB0	GPB1	GPB2	GPB3	GPB4	GPB5	GPB6	GPB7
100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115

Arduino C++ BLE Controller - Earle Philhower Library:

Debugging Bluetooth can be frustrating so I recommend you get your new keyboard working first over USB using Adafruit TinyUSB (see next section). Then you can modify the pins and key matrix in my "Pico_W_MacBookA1286_BLE_i2c_hw.ino" code either using AI or per the following steps:

You will need to load the Earle Philhower C++ BLE libraries into the Arduino IDE. Under Tools, USB Stack, select Pico SDK. Also under tools, IP/Bluetooth Stack, select IPv4 + Bluetooth.

Update the number of row and column pins:

#define NUM_ROWS 16

#define NUM_COLS 9

List the row and column pins:

const uint8_t row_pins[NUM_ROWS] = {

101, 20, 112, 9, 102, 10, 103, 19, 111, 11, 110, 18, 105, 14, 107, 15

};

const uint8_t col_pins[NUM_COLS] = {

100, 7, 113, 8, 17, 109, 16, 106, 108

};

Update the key matrix table shown below. Note the letters, numbers, and punctuation are inside single quotes but other keys use KEY_xxx. Backspace and quote are special:

'\\' this is the backspace key

'\'' this is the single quote key

```
const uint8_t keymap[NUM_ROWS][NUM_COLS] = {
//  GPA0=100    7          GPB5=113    8           17            GPB1=109       16            GPA6=106        GPB0=108
  { 'q',        '1',       'a',        'z',        KEY_F11,      KEY_BACKSPACE, KEY_F12,      KEY_F1,         0 },          //GPA1=101
  { 'w',        '2',       's',        'x',        KEY_ESC,      KEY_TAB,       '`',          KEY_F2,         0 },          //20
  { 'e',        '3',       'd',        'c',        0,            0,             0,            KEY_F3,         0 },          //GPB4=112
  { 'y',        '6',       'h',        'n',        '/',          ' ',           '=',          KEY_F6,         0 },          //9
  { 'r',        '4',       'f',        'v',        0,            0,             ']',          KEY_F4,         0 },          //GPA2=102
  { 0,          0,         0,          0,          KEY_RETURN,   0,             0,            KEY_RIGHT_SHIFT, 0 },         //10
  { 0,          0,         0,          KEY_LEFT_GUI, '0',        0,             KEY_RIGHT_GUI, 0,             0 },          //GPA3=103
  { 't',        '5',       'g',        'b',        '-',          0,             '[',          KEY_F5,         0 },          //19
  { 0,          0,         0,          0,          KEY_F10,      KEY_RIGHT_ALT, 0,            0,              0 },          //GPB3=111
  { 0,          0,         0,          0,          'p',          KEY_CAPS_LOCK, 0,            0,              0 },          //11
  { 'u',        '7',       'j',        'm',        ';',          '\'',          0,            KEY_F7,         0 },          //GPB2=110
  { 'i',        '8',       'k',        ',',        0,            KEY_LEFT_ARROW, KEY_RIGHT_ARROW, KEY_F8,      0 },          //18
  { 'o',        '9',       'l',        '.',        '\\',         KEY_DOWN_ARROW, KEY_UP_ARROW,  KEY_F9,        0 },          //GPA5=105
  { 0,          0,         0,          0,          0,            0,             0,            0,              KEY_LEFT_SHIFT }, //14
  { 0,          0,         0,          0,          0,            0,             0,            0,              KEY_LEFT_ALT },   //GPA7=107
  { 0,          0,         0,          0,          0,            0,             0,            0,              KEY_LEFT_CTRL }   //15
};
```

Change the Bluetooth name associated with your keyboard:

KeyboardBLE.begin("MacBook Pro HID", "Apple Wireless Mod");

Any compiler or operational errors can be fed to AI along with a link to the code and it will tell you what to fix. If you're having trouble getting Bluetooth to connect to your PC or phone, (but it worked previously), remember that each time you re-flash the Pico, you need to "Remove Device" in your PC or phone's Bluetooth settings and then pair it again.

If you want the media keys to work, look at lines 158 through 194. Modify the row and column index numbers so it can detect when the Fn key is held down and when the desired Function key is pressed.

Note: This BLE code controls the MCP23018 port expander using the Pico's built in I2C hardware. I initially had Bluetooth timing issues at power up that were resolved by using bit-bang I2C code. That's no longer the case with the hardware I2C code (thanks Claude), but I'll leave the bit-bang link just in case it helps someone.

Arduino C++ USB Controller – Adafruit TinyUSB Library:

If you haven't already done so, load the Earle Philhower board package and the Adafruit TinyUSB library into the Arduino IDE. Modify the MacBook code Pico_MacBookA1286_USB_i2c_hw.ino with your keyboard's matrix and row/column pins, as detailed below. Note: This task could be done by AI.

// Row and Column count

const int NUM_ROWS = 16; // number of row signals

const int NUM_COLS = 9; // number of column signals

// Row pins stored in an array

const int rowPins[NUM_ROWS] = {101, 20, 112, 9, 102, 10, 103, 19, 111, 11, 110, 18, 105, 14, 107, 15};

// Column pins stored in an array

const int colPins[NUM_COLS] = {100, 7, 113, 8, 17, 109, 16, 106, 108};

// Base Keyboard Map. A zero indicates no key at that position.

```
const uint8_t keyMap[NUM_ROWS][NUM_COLS] = {
//  GPA0=100    7           GPB5=113    8               17              GPB1=109          16                  GPA6=106            GPB0=108
  { HID_KEY_Q,  HID_KEY_1,  HID_KEY_A,  HID_KEY_Z,      HID_KEY_F11,    HID_KEY_BACKSPACE, HID_KEY_F12,        HID_KEY_F1,         0 },          //GPA1=101
  { HID_KEY_W,  HID_KEY_2,  HID_KEY_S,  HID_KEY_X,      HID_KEY_ESCAPE, HID_KEY_TAB,       HID_KEY_GRAVE,      HID_KEY_F2,         0 },          //20
  { HID_KEY_E,  HID_KEY_3,  HID_KEY_D,  HID_KEY_C,      0,              0,                 0,                  HID_KEY_F3,         0 },          //GPB4=112
  { HID_KEY_Y,  HID_KEY_6,  HID_KEY_H,  HID_KEY_N,      HID_KEY_SLASH,  HID_KEY_SPACE,     HID_KEY_EQUAL,      HID_KEY_F6,         0 },          //9
  { HID_KEY_R,  HID_KEY_4,  HID_KEY_F,  HID_KEY_V,      0,              0,                 HID_KEY_BRACKET_RIGHT, HID_KEY_F4,      0 },          //GPA2=102
  { 0,          0,          0,          0,              HID_KEY_ENTER,  0,                 0,                  HID_KEY_SHIFT_RIGHT, 0 },         //10
  { 0,          0,          0,          HID_KEY_GUI_LEFT, HID_KEY_0,    0,                 HID_KEY_GUI_RIGHT,  0,                  0 },          //GPA3=103
  { HID_KEY_T,  HID_KEY_5,  HID_KEY_G,  HID_KEY_B,      HID_KEY_MINUS,  0,                 HID_KEY_BRACKET_LEFT, HID_KEY_F5,       0 },          //19
  { 0,          0,          0,          0,              HID_KEY_F10,    HID_KEY_ALT_RIGHT, 0,                  0,                  0 },          //GPB3=111
  { 0,          0,          0,          0,              HID_KEY_P,      HID_KEY_CAPS_LOCK, 0,                  0,                  0 },          //11
  { HID_KEY_U,  HID_KEY_7,  HID_KEY_J,  HID_KEY_M,      HID_KEY_SEMICOLON, HID_KEY_APOSTROPHE, 0,              HID_KEY_F7,         0 },          //GPB2=110
  { HID_KEY_I,  HID_KEY_8,  HID_KEY_K,  HID_KEY_COMMA,  0,              HID_KEY_ARROW_LEFT, HID_KEY_ARROW_RIGHT, HID_KEY_F8,       0 },          //18
  { HID_KEY_O,  HID_KEY_9,  HID_KEY_L,  HID_KEY_PERIOD, HID_KEY_BACKSLASH, HID_KEY_ARROW_DOWN, HID_KEY_ARROW_UP,  HID_KEY_F9,       0 },          //GPA5=105
  { 0,          0,          0,          0,              0,              0,                 0,                  0,                  HID_KEY_SHIFT_LEFT },  //14
  { 0,          0,          0,          0,              0,              0,                 0,                  0,                  HID_KEY_ALT_LEFT },    //GPA7=107
  { 0,          0,          0,          0,              0,              0,                 0,                  0,                  HID_KEY_CONTROL_LEFT } //15
};
```

The above key matrix uses the key names required by the Adafruit TinyUSB library.

The Fn - media keys were coded for audio controls.

Note: I initially wrote code that bit-bangs the I2C bus to the MCP23018 port expander. It works but it's not as "pretty" as the new code.

Additional laptop keyboards that have been compiled with TinyUSB:

Lenovo E550

QMK USB Controller:

Note: QMK does not support the Raspberry Pi Pico 2 or any other RP2350 module.

QMK is widely considered the de facto open-source standard firmware for mechanical keyboards. There are lots of custom printed circuit boards for building a QMK based mechanical keyboard but very few for laptop keyboards. QMK has many features but the most important for this circuit board is the ability to control the MCP23018 port expander over the I2C bus. I'm just a beginner when it comes to QMK but I got lots of help from the AI assistant "Claude".

Before you dive into QMK, you must determine the key matrix and row/column pins as outlined above in the Keyboard Decoding section.

Read through the QMK tutorial and other web sites to get a feel for the build process and the command line interface (CLI). The CLI uses Python scripts so you'll need a standalone version of Python installed on your computer (not just the Python bundled inside Thonny). A good code editor like Notepad++ is also needed. Disregard talk about the browser based "QMK Configurator" as it's not useful for a laptop keyboard.

Download and run QMK-MSYS. Use its CLI to issue the command, "qmk setup". The installer will download a lot of files from GitHub including a folder full of mechanical keyboard files (that you won't use).

Use the pwd, ls, and cd Linux commands in the CLI to navigate to the following directory:

/c/Users/<user_name>

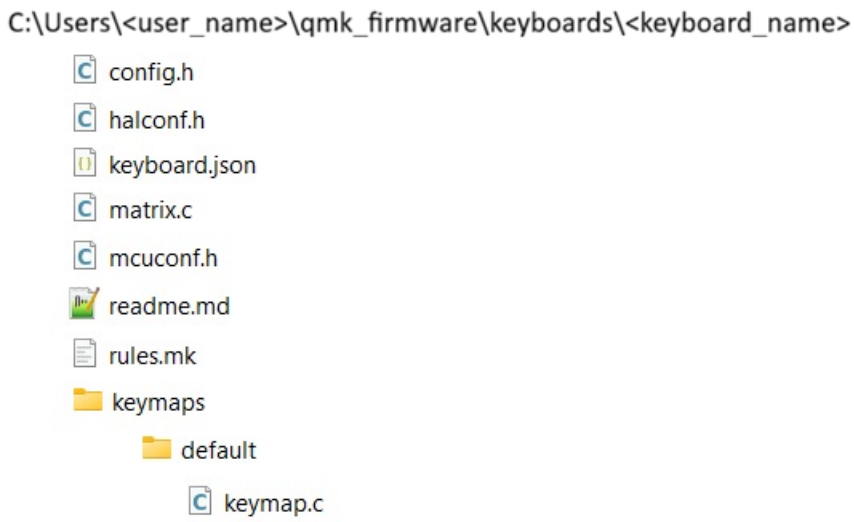
In the CLI, run the "qmk new-keyboard" command. This will create files and folders in the proper directory structure for later editing. It will ask for your new keyboard name, which can only have lower case letters, underscore, and numbers. You are going to completely rewrite the files it makes so the questions about your GitHub user name, real name, list of mechanical keyboards, list of boards, and list of controllers doesn't really matter. The main thing is that a new folder will be created in the keyboards directory with your keyboard name. On my Windows 11 PC, it was at:

/c/Users/<user_name>/qmk_firmware/keyboards/<keyboard_name>

Use Windows file explorer (or equivalent) to navigate to the new <keyboard_name> folder. It will have 2 files, keyboard.json and readme.md as well as one folder named keymaps. Go down into the keymaps folder and then down into the default folder. Replace the keymap.c file with the keymap.c from my repo. Navigate back up two levels and replace the keyboard.json file with the keyboard.json from my repo. Replace the readme.md file with the readme.md from my

repo. Copy the config.h, halconf.h, matrix.c, mcuconf.h, and rules.mk files from my repo to this directory. There should now be 7 files (and a keymaps folder) in your <keyboard_name> folder.

Instead of running the “qmk new-keyboard” command you can just make your own folders and copy the files from my repo into them. This is the directory structure and the files:



The tutorials tell you to build test code for one of the mechanical keyboards in order to see if you have all the files and tools properly set up. You can skip this or use the unedited files you just downloaded to do a QMK compile (it will take about 30 minutes). To compile the code, be located at <keyboard_name> and type the following in the CLI: qmk compile -kb <keyboard_name> -km default

Code Modifications: Instead of manually modifying each file, you could provide AI a link to my files and to your key matrix results and let it do the code modifications. The MacBook keyboard has a backlight so the QMK files control it using a PWM signal from Pico GP4. The following file changes will describe the modifications you need to make, including how to remove the backlight control.

keymap.c must be modified with the key matrix that you built for your keyboard using the manual method, Python code, or via AI. The Python code should be a perfect base layer. The AI method should be pretty close but the manual method will take a lot of editing. For the media keys that are activated when Fn is held down, determine where the associated function key is located in the _BASE matrix. Place the media key name in the equivalent spot in the _FN matrix.

keyboard.json modifications are fairly obvious. If you don’t have a keyboard backlight, delete the line

"backlight": true, (it defaults to false).

Also remove the backlight section:

```
  "backlight": {
    "driver": "pwm",
    "pin": "GP4",
    "levels": 8
  },
```

Change the matrix_size section with the number of rows and columns in your keyboard.

```
  "matrix_size": {
    "rows": 16,
    "cols": 9
  },
```

The layouts section must be modified to match the number of rows and columns in your keyboard (the pattern is obvious). If you only have 8 columns (instead of 9), remove the last line from each group. Here is the first group showing the line to delete.

```
{"matrix": [0, 0], "x": 0, "y": 0},
{"matrix": [0, 1], "x": 1, "y": 0},
{"matrix": [0, 2], "x": 2, "y": 0},
{"matrix": [0, 3], "x": 3, "y": 0},
{"matrix": [0, 4], "x": 4, "y": 0},
{"matrix": [0, 5], "x": 5, "y": 0},
{"matrix": [0, 6], "x": 6, "y": 0},
{"matrix": [0, 7], "x": 7, "y": 0},
~~{"matrix": [0, 8], "x": 8, "y": 0},~~
```

If your matix has more or less than 16 rows, add or delete groups at the end. The number of “matrix” lines should equal the number of rows times the number of columns. If you get this wrong, the QMK compiler will error out. It will also error if you put a comma on the last matrix line.

config.h only needs the two backlight #defines removed from the end, assuming your keyboard has no backlight.

halconf.h needs the HAL_USE_PWM define removed assuming you don't have a keyboard backlight.

matrix.c needs the row and column pins changed for your keyboard at these locations.

// ---- Keyboard specific pin lists ----

static const uint8_t row_pins[MATRIX_ROWS] = {101, 20, 112, 9, 102, 10, 103, 19, 111, 11, 110, 18, 105, 14, 107, 15};

static const uint8_t col_pins[MATRIX_COLS] = {100, 7, 113, 8, 17, 109, 16, 106, 108};

mcudonf.h needs the last two lines regarding the backlight PWM removed assuming you don't have a keyboard backlight.

rules.mk needs no modifications.

readme.md is not really needed but it's a good idea to modify it with your keyboard model number.

After editing the files described above, use the CLI to navigate to /c/Users/<user_name>/qmk_firmware/keyboards/<keyboard_name>

Compile the code in the CLI with the command:

qmk compile -kb <keyboard_name> -km default

The compile takes 30 minutes on my slow computer (once I clear the errors). The QMK source code is updated every 3 months so depending on what version you downloaded from their repo, you may need to change the syntax of the command s shown here.

Flashing your controller:

To load your new QMK code into a Raspberry Pi Pico, start with the USB disconnected, push and hold the button on the Pico while plugging in the USB cable. Release the button and a folder will show up as RPI-RPI2 with a drive letter. Drag the <keyboard_name>_default.uf2 file that was just compiled into the Pico folder. The .uf2 file is located at C:Users/<user_name>/qmk_firmware

The RPI-RPI2 folder will blink out and the keyboard should be functional.

There are two other methods to put the Pico into bootloader mode:

1. Double tap the reset button (assuming you wired one to the J1 connector on the board.
2. Hold the Q key down while plugging in the USB cable. This is defined as “bootmagic” in the code.

Additional laptop keyboards that have been compiled with QMK:

Lenovo E550

KMK USB Controller:

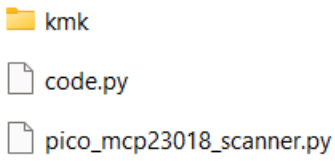
KMK usually has a single Python program named “code.py” loaded in the Pico drive but a second program called “pico_mcp23018_scanner.py” is needed to control the port expander. The code.py file at my repo has the keymap for a MacBook Pro A1286. Modify it with your keyboard’s information. The AI “Claude” wrote the scanner code and gave the following information:

On the CIRCUITPY drive, put code.py, pico_mcp23018_scanner.py and the kmk folder side by side. No Adafruit libraries are needed; the scanner talks to the MCP23018 directly with busio. It uses the same pin numbers as the QMK and Arduino code: 0–28 for Pico GPIOs, 100–107 for GPA0–7, and 108–115 for GPB0–7.

How the scanner works:

- Nothing is ever driven high. Only one row is driven low at a time, and every other row and column stays an input with a pull-up.
- Expander rows take one I2C write each way. The MCP23018’s output latches are set to 0 at startup. Driving an expander row low is a single direction-register write, and releasing it is another.
- One I2C read per row. All 16 expander pins come in one transfer. A full scan is 32 I2C transfers: 16 reads, plus a drive and a release write for each of the 8 expander rows.
- Debounce: a key must read the same on 2 scans in a row before its change is reported.
- I2C errors don’t leave anything stuck. The scanner releases all rows, keeps the last good key state, and sets up the expander again on the next scan.

I loaded the Pico with CircuitPython, then loaded KMK but didn’t load any Adafruit keyboard files. The files on the Pico CIRCUITPY drive should look like this:



I didn’t use Thonny for any debug (it worked first time).